

Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

## Tarea 2

# Fundamentos de Java: Tipos de Datos, Arreglos, Modularidad y Validaciones



**FIUSAC**  
FACULTAD DE INGENIERÍA  
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



**PONDERACIÓN: 1 pts**

 **Tiempo estimado: 4 hrs**

## 1. MARCO FORMATIVO

### 1.1. Valores

| Nombre del valor | ¿Cómo se aplica en tu laboratorio?                                                                                               |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad  | El estudiante demostrará disciplina al validar correctamente los datos ingresados por el usuario, evitando errores por descuido. |
| Honestidad       | El estudiante documentará y versionará su trabajo en Git de forma transparente, reflejando el proceso real de desarrollo.        |

### 1.2. Competencia(s)

| Tipo de Competencia    |                                                                                                                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Competencia General    | Comunica soluciones técnicas complejas a audiencias diversas utilizando documentación técnica, presentaciones y demostraciones interactivas, adaptando el nivel de detalle según el contexto y audiencia objetivo |
| Competencia Específica | Analiza los tipos de datos y estructuras de control de Java para construir programas que validen correctamente la entrada del usuario, aplicando control de versiones con Git durante el proceso                  |

### 1.3. Objetivo SMART

| SMART                        | Definición                                      | Objetivo redactado                                                                                                                                                       |
|------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Específico</b><br>(¿Qué?) | El objetivo es concreto y tangible.             | Los estudiantes serán capaces de crear programas en Java que declaren correctamente tipos de datos, validen la entrada por consola y versionen su código utilizando Git. |
| <b>Medible</b><br>(¿Cuánto?) | El objetivo tiene una medida objetiva de éxito. | Lograr un 100% de efectividad al validar los datos de entrada y un historial de commits completo en el repositorio.                                                      |

|                              |                                                                               |                                                                                                                            |
|------------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Alcanzable (¿Cómo?)</b>   | El objetivo debe ser posible con los recursos disponibles.                    | Utilizando únicamente las herramientas de laboratorio disponibles en clase (Java, Git y un editor de código).              |
| <b>Realista (¿Para qué?)</b> | El objetivo contribuye a metas más amplias.                                   | Esto ayudará a desarrollar habilidades de programación, control de versiones y buenas prácticas de desarrollo de software. |
| <b>A Tiempo (¿Cuándo?)</b>   | El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso | Cumplir este aprendizaje en un plazo máximo de 2 día(s).                                                                   |

## 2. Material de Apoyo

- Lenguaje de programación Java
- Control de versiones con Git (comandos básicos: init, add, commit, push, log)
- Tipos de datos primitivos en Java (int, double, char, boolean, String, etc.)
- Declaración y uso de variables
- Comentarios de una línea (`//`) y multilínea (`/* */`)
- Métodos (modularidad)
- Arreglos de una dimensión y de dos dimensiones
- Clase Scanner para entrada por consola
- Validación de datos (condicionales, bucles de validación)

### Restricción importante

No se permite el uso de ninguna clase del paquete `java.util` relacionada a Collections (ArrayList, List, LinkedList, HashMap, HashSet, etc.). Todo almacenamiento de datos debe realizarse únicamente con arreglos (arrays).

## 3. Actividad

Crear un repositorio en Git para versionar el ejercicio, e implementar en Java un programa de consola que cumpla con lo siguiente:

1. **Tipos de datos y variables:** Declarar variables utilizando al menos 4 tipos de datos primitivos distintos (int, double, char, boolean, etc.), inicializándolas correctamente según su naturaleza.
2. **Comentarios:** Incluir en el código al menos un comentario de una línea y un comentario multilínea, explicando la lógica de secciones relevantes del programa.

3. **Modularidad (métodos):** Dividir la lógica del programa en al menos 4 métodos distintos, cada uno con una responsabilidad clara (por ejemplo: un método para leer datos, otro para validar, otro para procesar un arreglo, otro para mostrar resultados).
4. **Entrada por consola y validación:** Utilizando la clase `Scanner`, solicitar al usuario el ingreso de varios datos (por ejemplo, un conjunto de números). El programa debe:
  - Validar que el dato ingresado sea del tipo correcto (por ejemplo, rechazar texto donde se espera un número).
  - Validar que el dato esté dentro de un rango definido por el estudiante (por ejemplo, entre 1 y 100).
  - Volver a solicitar el dato hasta que sea válido, sin cerrar el programa abruptamente.
5. **Arreglo de una dimensión:** Almacenar los datos ingresados y validados en un arreglo de una dimensión. Implementar un método que recorra el arreglo y muestre, por ejemplo, el valor máximo, mínimo y el promedio.
6. **Arreglo de dos dimensiones:** Solicitar al usuario datos adicionales para llenar una matriz (arreglo de dos dimensiones), validando también su entrada. Implementar un método que recorra la matriz y muestre su contenido de forma ordenada (por filas y columnas), además de algún cálculo simple (por ejemplo, suma de todos los elementos o suma por fila).
7. **Versiónamiento:** Realizar al menos 5 commits en el repositorio de Git, reflejando avances lógicos del desarrollo (por ejemplo: estructura inicial y variables, métodos y validaciones, arreglo unidimensional, arreglo bidimensional, versión final).

El estudiante deberá entregar:

- El enlace al repositorio de Git con el historial de commits.
- Todo el código deberá estar dentro de una carpeta llamada **Tarea2** dentro del repositorio.
- El nombre del repositorio debe de ser **IPC1X\_Tareas\_NombreApellido\_Carné** (la X debe reemplazarse por la sección)
- El archivo .java con el código completo del programa.
- Capturas de pantalla que evidencien el funcionamiento correcto del programa, incluyendo casos de validación exitosos y fallidos, y la salida de los arreglos de una y dos dimensiones.

Todo debe entregarse en un documento PDF. (**link del repositorio, capturas de ejecución**)

## 4. Cronograma

|                     |            |
|---------------------|------------|
| Asignación de tarea | 01/08/2026 |
| Entrega de Tarea    | 03/08/2026 |

## 5. Rúbrica de calificación

| CRITERIO                  | DESCRIPCIÓN                                                                      | PUNTEO         |
|---------------------------|----------------------------------------------------------------------------------|----------------|
| <b>Uso de Git</b>         | Repositorio creado con historial de commits que refleja el avance del desarrollo | 20 pts         |
| <b>Tipos de datos</b>     | Declaración correcta de los tipos de datos según cada dato solicitado            | 20 pts         |
| <b>Validaciones</b>       | El programa valida correctamente la entrada y no se cierra ante datos inválidos  | 20 pts         |
| <b>Entrada en consola</b> | Uso correcto de la clase Scanner para solicitar los datos                        | 20 pts         |
| <b>Entregable PDF</b>     | Entregable correcto y completo                                                   | 20 pts         |
| <b>TOTAL</b>              |                                                                                  | <b>100 pts</b> |

### Reflexión y aplicación práctica

¿Qué aprendí con esta tarea?

¿Cómo crees que aplicarás este conocimiento en otro curso o en la vida profesional?

